

SemanticPromptTransfer v0.19 패키지 요구사항 정의서

L0 기본 운영 RAG 패키지와 STX엔진 축약 예제

2026-08-30

차 례

1 문서 목적	2
2 패키지 경계	2
2.1 포함 범위	2
2.2 제외 범위	2
3 PipelineConfig	2
4 DocumentScope	3
5 PackageChunkBuilder	3
5.1 본문 청크	3
5.2 표 청크 L0	3
5.3 L1·L2 실험 옵션	4
6 E5OnnxEncoder	4
7 RAGIndex	4
7.1 인덱스 구성	4
7.2 문서 UPSERT	4
8 RetrievalEngine	5
8.1 범위 필터	5
8.2 Dense와 BM25	5
9 PromptPackageBuilder	5
10 RAGPipeline	5
10.1 오프라인 상태 전이	5
10.2 온라인 상태 전이	5
11 OfflineIndexBuilder와 OnlineRAGService	6
12 CLI	6
13 STX엔진 예제	6
14 검증 계약	6
14.1 STX 회귀	6
14.2 패키지 테스트	7
15 운영 비기능 요구사항	7
16 알려진 제한과 다음 우선순위	7
17 요건정의서 출력 규격	7

1 문서 목적

이 문서는 “각 클래스별로 어떻게 산출했는지 수학적으로 설명해 달라”는 질문에 답할 수 있도록 SemanticPromptTransfer v0.19 패키지의 입력, 변환식, 공개 계약, 운영 경계와 검증 결과를 정의한다.

v0.19는 Cell 1~3을 수정하거나 실행하지 않는다. 기존 v0.17 MASTER를 입력으로 사용하고 Cell 4~7만 설치 가능한 Python 패키지로 제공한다.

$$train_{1,3} = 0, \quad write(MASTER) = 0, \quad source(Cell4) = MASTER_{v0.17}.$$

운영 기본 표현은 L0이다.

$$L_{default} = 0, \quad L \in \{1, 2\} \Rightarrow explicit \ opt-in.$$

현재 L0는 완전한 원시 평문이 아니라 표의 행·열 문맥을 선형화한 semantic_linearized 표현이다. L1·L2는 실험 옵션으로 유지한다.

2 패키지 경계

2.1 포함 범위

단계	클래스	책임
Cell 4	PackageChunkBuilder	MASTER를 L0 RAG 청크로 변환
Cell 5	E5OnnxEncoder	CPU 임베딩 생성
저장	RAGIndex	청크·벡터·운영 메타데이터 저장
Cell 6	RetrievalEngine	Dense·BM25·RRF 검색과 범위 필터
Cell 7	PromptPackageBuilder	출처 추적 프롬프트 생성
통합	RAGPipeline	빌드·로드·검색·프롬프트 수명주기
운영	OfflineIndexBuilder	오프라인 인덱스 구축
운영	OnlineRAGService	인덱스 1회 로드 후 반복 질의
인터페이스	spt-rag	index/retrieve/prompt/inspect CLI

2.2 제외 범위

- PDF 추출, 계층 판정, 표 연속성, 헤더·구분자·값 모델
- v0.17 MASTER와 저장 모델의 재생성 또는 재학습
- 전체 STX엔진 PDF와 60MB MASTER
- E5 ONNX 모델 파일
- 답변 생성 LLM 호출

대용량 자료는 패키지에 중복 포함하지 않고 경로·버전·SHA-256으로 연결한다.

3 PipelineConfig

PipelineConfig는 운영 불변식과 선택 가능한 실행 모드를 한 객체에 고정한다.

$$C = (l, m_M, m_I, p_I, s_I, M, o, b, k, B),$$

여기서 l 은 표현 수준, m_M 은 MASTER 모드, m_I 은 인덱스 모드, p_I 는 인덱스 경로, s_I 는 쓰기 전략, M 은 청크 길이, o 는 중첩, b 는 배치 크기, k 는 검색 수, B 는 프롬프트 예산이다.

기본값은 다음과 같다.

변수	기본값
표현 수준 l	0
MASTER 모드	LOAD

변수	기본값
인덱스 모드	MEMORY
최대 청크 문자 M	1,800
본문 중첩 o	180
배치 b	24
Top-k k	5
프롬프트 문자 예산 B	24,000

운영 팩토리는 역할을 명시한다.

$$C_{build} : m_I = WRITE, \quad C_{serve} : m_I = LOAD.$$

Cell 1~3 동결을 보장하기 위해 $m_M \neq LOAD$ 는 즉시 오류로 처리한다.

4 DocumentScope

한 문서의 모든 청크에는 다음 격리 메타데이터를 결합한다.

$$S = (tenant, case, document, financial_scope, version, tags).$$

청크 K_i 의 최종 메타데이터는 다음과 같다.

$$meta'(K_i) = meta(K_i) \cup S.$$

tenant_id, case_id, document_id는 필수다. 온라인 서비스는 최소 tenant_id와 case_id 필터가 없는 요청을 거부하여 다른 사용자나 심사건의 근거가 섞이지 않도록 한다.

CLI의 retrieve와 prompt 명령도 같은 이유로 --tenant-id와 --case-id를 필수 인자로 요구한다.

5 PackageChunkBuilder

5.1 본문 청크

MASTER의 본문 블록 집합을 $B = \{b_1, \dots, b_n\}$ 라 한다. 머리말·꼬리말, 계층 제목, 표 내부, 표제목·단위·주석은 본문 후보에서 제외한다.

$$B' = B - (H \cup F \cup T \cup U \cup N).$$

연속 블록은 동일한 scope_heading_id와 길이 제한을 만족할 때 결합한다.

$$K_j = \text{join}(b_a, \dots, b_c), \quad \text{scope}(b_a) = \dots = \text{scope}(b_c), \quad \text{len}(K_j) \leq M.$$

동일 계층에서 길이로 분할하면 $o = 180$ 자의 중첩을 사용한다. 현재 표는 별도 청크이지만 본문 절단 이벤트는 아니므로 표 앞뒤 본문이 동일 계층이면 한 청크에 포함될 수 있다. 이는 v0.19의 알려진 개선 항목이다.

5.2 표 청크 L0

표의 행 경로 r_i , 열 경로 c_j , 값 v_{ij} 와 단위 u 를 선형화한다.

$$R_0(T) = \text{title} \oplus \text{unit} \oplus \sum_{(i,j) \in V} (r_i \oplus c_j \oplus \oplus v_{ij}).$$

표제목·단위·행열 문맥과 원문 출처는 유지하지만 JSON 계층은 기본 검색 문서에 넣지 않는다. L0의 embedding_text와 document는 동일하다.

5.3 L1·L2 실험 옵션

L1 은 JSON 외피와 Markdown 표를, L2 는 열·행·셀 계층을 추가한다. 운영 경로에서는 명시적인 `representation_level`이 없으면 생성하지 않는다.

$$R_1 = R_0 \cup \{\text{markdown}, \text{units}, \text{notes}\}, \quad R_2 = R_1 \cup \{\text{rows}, \text{columns}, \text{cells}, \text{spans}\}.$$

6 E5OnnxEncoder

기본 인코더는 `intfloat/multilingual-e5-small` ONNX INT8 CPU 모델이며 출력 차원은 $d = 384$ 다. 문서에는 `passage:`, 질의에는 `query:`를 붙인다.

토큰 구간 x_j 의 은닉벡터 H_j 와 마스크 a_j 를 평균 풀링한다.

$$v_j = \frac{\sum_t a_{jt} H_{jt}}{\sum_t a_{jt}}, \quad \hat{v}_j = \frac{v_j}{\|v_j\|_2}.$$

긴 입력은 최대 512토큰과 32토큰 겹침으로 나누며 구간 평균 z 를 다시 정규화한다.

$$z = J^{-1} \sum_{j=1}^J \hat{v}_j, \quad e(x) = \frac{z}{\sqrt{z^T z}}.$$

EncoderBackend 계약을 구현하면 향후 Qwen 또는 내부 모델로 교체할 수 있다. 인덱스 로드 시 모델 ID, 모델 SHA-256, 차원이 빌드 시점과 같은지 검증한다.

7 RAGIndex

7.1 인덱스 구성

체크 집합 $K = (K_1, \dots, K_N)$ 과 정규화 임베딩 행렬을 저장한다.

$$E = [e(K_1); \dots; e(K_N)] \in \mathbb{R}^{N \times d}.$$

현재 기준 백엔드는 압축 NPZ다. 체크 JSON, E , 인덱스 메타데이터를 한 파일에 저장하고 `allow_pickle=False`로 읽는다.

쓰기 과정은 임시 파일을 완성한 뒤 원자적으로 교체한다.

$$I_{new} \rightarrow I_{tmp} \xrightarrow{\text{atomic replace}} I_{target}.$$

7.2 문서 UPSERT

새 인덱스가 포함하는 문서 키를

$$D_{new} = \{(\text{tenant}, \text{case}, \text{document})\}$$

라 하면 기존 인덱스에서 같은 문서의 체크를 제거한 뒤 새 체크를 결합한다.

$$I' = \{K_i \in I : \text{key}(K_i) \notin D_{new}\} \cup I_{new}.$$

동시 업데이트는 `.lock` 파일의 프로세스 잠금으로 직렬화한다. 표현 수준 또는 인코더 해시가 다른 인덱스는 병합하지 않는다.

8 RetrievalEngine

8.1 범위 필터

질의 범위 필터 F 를 먼저 적용한다.

$$\mathcal{E}_F = \{i : \forall (k, v) \in F, v \in \text{meta}_i[k]\}.$$

Dense와 BM25 순위는 $\mathcal{E}_F$ 안에서만 계산되므로 다른 심사건의 청크는 후보 순위에도 들어가지 않는다.

8.2 Dense와 BM25

정규화된 벡터의 코사인 유사도는 내적이다.

$$s_d(i) = e(Q)^T e(K_i).$$

BM25는 한국어·영문 단어와 문자 2-gram을 사용한다.

$$s_b(i, Q) = \sum_{t \in Q} IDF(t) \frac{tf(t, K_i)(k_1 + 1)}{tf(t, K_i) + k_1(1 - b + b|K_i|/avgdl)}.$$

두 순위를 RRF로 결합한다.

$$s(i) = \frac{1}{60 + r_d(i)} + \frac{1}{60 + r_b(i)} + 0.0015I_{table} + 0.0007I_{unit}.$$

결과에는 점수, 순위, 출처, 적용 필터, 대상 청크 수, 표현 수준, trace_id, 지연시간을 기록한다.

9 PromptPackageBuilder

검색 결과 $E^*(Q)$ 를 공급자 독립적인 프롬프트로 변환한다.

$$P = \mathcal{B}(Q, E^*(Q), B, C),$$

C 는 근거 외 사실 금지, 수치·기간·단위 보존, 모든 핵심 주장에 evidence ID 표시 조건이다. 각 근거에는 청크 ID, 논리표 ID, 페이지와 검색 점수가 포함된다.

L0에서 STX 기준 Top-5는 24,000자 예산 안에 모두 전달된 것을 v0.18에서 확인했다.

10 RAGPipeline

10.1 오프라인 상태 전이

$$MASTER \xrightarrow{ChunkBuilder} K \xrightarrow{Encoder} E \xrightarrow{RAGIndex.save} I.$$

WRITE는 인덱스를 생성하고, UPSERT는 동일 문서를 교체하면서 다른 문서를 유지한다.

10.2 온라인 상태 전이

$$I \xrightarrow{load\ once} (K, E, BM25) \xrightarrow{query} (TopK, trace, latency).$$

온라인 프로세스는 요청마다 MASTER를 읽거나 문서를 재임베딩하지 않는다. 모델 세션, 벡터 행렬, BM25를 프로세스 시작 시 한 번 생성해 재사용한다.

health()는 준비 상태, 패키지 버전, 표현 수준, 인덱스 모드, 청크 수와 uptime을 반환한다.

11 OfflineIndexBuilder와 OnlineRAGService

OfflineIndexBuilder는 WRITE 구성만 허용하고 MASTER와 DocumentScope를 받아 인덱스를 생성한다.

OnlineRAGService는 LOAD 구성만 허용하며 start() 이후 search()와 prompt()를 반복 호출한다. 온라인 호출에는 tenant_id와 case_id가 필요하다.

운영 권장 배치는 다음과 같다.

1. 업로드 워커: PDF 처리 및 MASTER 생성(Cell 1~3 외부 시스템)
2. 인덱스 워커: Cell 4~6 오프라인 수행, 문서 UPSERT
3. 질의 API 워커: 인덱스와 E5를 1회 로드
4. 답변 워커: Cell 7 프롬프트를 선택한 LLM에 전달

현재 NPZ 정확도검사는 STX 규모와 단일·소규모 심사건의 기준 백엔드다. 다수 사용자·대규모 문서에서는 동일한 인덱스 계약 뒤에 Chroma 또는 별도 Vector DB 백엔드를 추가한다.

12 CLI

CLI는 다음 명령을 제공한다.

명령	역할
spt-rag index	MASTER를 L0 인덱스로 구축·UPSERT
spt-rag retrieve	저장 인덱스에서 Top-k 검색
spt-rag prompt	검색 후 근거 프롬프트 생성
spt-rag inspect	모델 없이 인덱스 메타데이터 확인

모든 명령의 표현 수준 기본값은 0이다.

13 STX엔진 예제

wheel에는 전체 보고서 대신 다음 축약 자료를 포함한다.

- L0 형식의 짧은 예제 청크 5개
- 기본 여신심사 쿼리 5개
- v0.18 전체 시험의 기대 Top-1 물리표 ID
- 외부 전체 MASTER를 사용하는 index/retrieve 명령 예시

축약 예제는 패키지 형식과 사용법 확인용이다. 검색 성능 검증은 외부 전체 MASTER와 동일 E5 모델로 수행한다.

전체 STX엔진 보고서는 패키지의 운영 입력이 아니라 예제 원천 자료다. 배포 wheel에는 짧은 사실형 샘플만 포함하며, 전체 PDF·MASTER·저장 모델·실제 심사건 인덱스는 별도 접근통제 저장소에서 관리한다.

14 검증 계약

14.1 STX 회귀

패키지 L0와 v0.18 L0를 전체 MASTER로 비교한 결과는 다음과 같다.

항목	결과
청크 수	1,090 / 1,090
청크 ID	완전 동일
임베딩 입력문	완전 동일
LLM 전달 문서	완전 동일
결합 SHA-256	완전 동일(아래 전체 해시)

결합 SHA-256은 다음 두 줄을 연결한 값이다.

4bce8e02700798b73a0803d8f37d5e1

d801d6d3efd7fe289e04fc0b35fd873ad

14.2 패키지 테스트

다음 항목을 자동 검증한다.

1. 기본 표현 수준 L0와 MEMORY 모드
2. L1·L2 명시 활성화
3. MASTER→청크→검색
4. tenant 필터 격리
5. 인덱스 WRITE/LOAD 왕복
6. 문서 UPSERT 교체와 다른 문서 유지
7. 온라인 tenant/case 필수 범위
8. wheel 설치 후 CLI 및 STX 예제 리소스 접근

15 운영 비기능 요구사항

영역	v0.19 보장	후속 확장
격리	tenant/case/document 메타데이터와 선필터	접근 토큰과 서버 권한 연계
일관성	원자적 저장, 프로세스 잠금, 해시 검증	분산 트랜잭션
성능	모델·인덱스 1회 로드, CPU E5	GPU/서빙 배치, ANN
관측성	trace ID, 검색 지연, health	중앙 로그·메트릭·알람
확장성	NPZ 정확검색 기준 구현	Chroma/Vector DB 백엔드
재현성	패키지·MASTER·모델·인덱스 버전	배포 레지스트리
개인정보	온라인 범위 필터, 원문 비로그 설계	암호화·보존기간·삭제 API

16 알려진 제한과 다음 우선순위

1. 본문 청크가 동일 계층에서 표 앞뒤를 건널 수 있다.
2. NPZ 인덱스는 문서 삭제 API와 분산 쓰기를 제공하지 않는다.
3. 금융 범위 consolidated/separate는 메타데이터 필터만 제공하며 자동 판별은 아직 없다.
4. 구조화 L2는 길이 증가 때문에 운영 기본으로 사용하지 않는다.
5. 답변 LLM의 수치·단위 정확도 평가는 패키지 외부 단계다.
6. 대규모 운영에서는 Chroma 또는 Vector DB 백엔드와 서비스 인증이 필요하다.

v0.19의 승인 기준은 “기본 L0 RAG 패키지를 재현 가능하게 설치하고, 오프라인 인덱싱과 범위가 격리된 온라인 검색을 분리 실행할 수 있음”이다.

17 요건정의서 출력 규격

PDF와 DOCX는 compact_reference_guide를 기준으로 하되, 한국어 수식 문서라는 목적에 맞춰 다음 이름 있는 출력 예외를 적용한다.

- A4 세로, 사방 22 mm 여백
- 본문과 표: NanumGothic, 수식: Latin Modern Math
- 본문 10.5 pt, 1.25줄, 제목·소제목 간격 고정
- 표 너비 9,360 DXA, 들여쓰기 120 DXA, 고정 열 너비와 반복 헤더
- 머리말·꼬리말과 페이지 번호 유지

최종 PDF와 DOCX는 모든 페이지를 이미지로 렌더링하여 한글·영문·수식 글리프, 줄바꿈, 표 경계, 머리말·꼬리말의 찢림과 겹침이 없는지 확인한다.